

ISP 위반 사례와 해결책

객체지향설계와 패턴

2026.03.24(Tue)



인터페이스 분리 원칙

- Interface Segregation Principle
- 정의
 - "클라이언트는 자신이 사용하지 않는 메서드에 의존 관계를 맺으면 안 된다."
 - 어떤 클래스가 자신이 사용하지 않는 메소드를 억지 구현하거나 비워 두는 것은 문제가 된다는 의미
 - 이 원칙은 범용(General-Purpose) 인터페이스가 아닌 좁고 세분화된(Narrow and Targeted) 인터페이스를 만드는 것을 독려하고 있습니다.
 - 전문화된 인터페이스가 필요하다는 의미로 이것도 되고 저것도 되는 인터페이스를 설계해서는 안된다.

인터페이스 분리 원칙의 위반과 해결책

- 여러 사례가 있는데 그 중에서 제일 쉬운 사례가 핸드폰 기능 중에 일정 관리나 수학함수 계산등이 안되는 예전 피쳐폰(Feature Phone)과 그런 기능을 비록 앱의 형태이긴 해도 기초적으로 제공할 수 있는 스마트폰으로 나눌 수 있는데 이 두개의 설계를 위해 어떤 실수가 있는지 살펴보자.
- 이 사례는 객체지향프로그래밍의 교재였던 "명품 자바 프로그래밍(4판)"에 소개된 5장 인터페이스 관련 내용을 이용하였다.

ISP의 위반 사례

```
interface PhoneInterface { // 인터페이스 선언
    final int TIMEOUT = 10000; // 상수 필드 선언
    void sendCall();           // 추상 메소드
    void receiveCall();        // 추상 메소드
    default void printLogo() { // default 메소드
        System.out.println("** Phone **");
    }
}

interface MobilePhoneInterface extends PhoneInterface { // 인터페이스 상속
    void sendSMS();
    void receiveSMS();
    // MobilePhoneInterface 일정 및 수학함수 계산기가 포함된다고 하자
    public int calculate(int x, int y);
}

interface MP3Interface { // 인터페이스 선언
    public void play();
    public void stop();
}
```

ISP의 위반 사례

```
class FeaturePhone implements MobilePhoneInterface {
```

```
    @Override
```

```
    public void sendCall() {  
        System.out.println("띵동띵동띵동");  
    }
```

```
    @Override
```

```
    public void receiveCall() {  
        System.out.println("땡땡땡.");  
    }
```

```
    @Override
```

```
    public void sendSMS() {  
        System.out.println("봍");  
    }
```

```
    @Override
```

```
    public void receiveSMS() {  
        System.out.println("봍봍봍봍봍");  
    }
```

```
    @Override
```

```
    public int calculate(int x, int y) {  
        // 일정 계산 기능을 구현하지 못한다면 이게 최선일 수 있다.  
        // 하지만 클래스의 구조에서는 이러면 이게 문제가 된다는 점  
        return 0;  
    }  
}
```

LG OMNIA와 삼성 Galaxy 스마트폰 이전의 피쳐폰(Feature Phone)에는 모바일 기능만 있고 날짜나 기타 수학 계산 기능은 없는 경우가 많았다.

ISP의 위반 사례 해결

```
class SmartPhone implements MobilePhoneInterface, MP3Interface {
```

```
// MobilePhoneInterface의 추상 메소드 구현
```

```
@Override
```

```
public void sendCall() {  
    System.out.println("따르릉따르릉~~");  
}
```

```
@Override
```

```
public void receiveCall() {  
    System.out.println("전화 왔어요.");  
}
```

```
@Override
```

```
public void sendSMS() {  
    System.out.println("문자갑니다.");  
}
```

```
@Override
```

```
public void receiveSMS() {  
    System.out.println("문자왔어요.");  
}
```

```
@Override
```

```
public int calculate(int x, int y) {
```

```
// 일정 계산 기능은 아니지만 일단 이렇다고 가정하자
```

```
    return x + y;
```

```
}
```

```
// MP3Interface의 추상 메소드 구현
```

```
@Override
```

```
public void play() {  
    System.out.println("음악 연주합니다.");  
}
```

```
@Override
```

```
public void stop() {  
    System.out.println("음악 중단합니다.");  
}
```

```
// 추가로 작성한 메소드
```

```
public void schedule() {  
    System.out.println("일정 관리합니다.");  
}
```

```
}
```

ISP의 위반 사례

```
public class InterfaceEx {  
    public static void main(String [] args) {  
        SmartPhone phone = new SmartPhone();  
        phone.printLogo();  
        phone.sendCall();  
        phone.play();  
        System.out.println("3과 5를 더하면 " + phone.calculate(3,5));  
        phone.schedule();  
    }  
}
```



해결책

- MobilePhoneInterface에 있는 calculate() 메소드를 SmartPhone 클래스에서는 구현해야 하지만 FeaturePhone 클래스에서는 구현할 필요가 없다.
- 이 calculate() 메소드를 PDA 기능이라고 할 수 있으니 별도의 클래스로 구현하자.
 - PDA(Personal Digital Assistant)는 터치스크린 기반의 휴대용 개인 정보 단말기로 스마트폰의 전신이다. 일정 관리, 메모, 연락처 저장 등의 기능이 있고 현재는 바코드 스캐너가 포함된 산업용이나 물류 현장에서 주로 사용된다.
 - 주로 가스 검침원분들이 들고 다니면서 도시가스 및 배기가스 유출 여부와 확인 처리를 즉시 처리하고 있다.

ISP의 위반 사례 해결

```
interface PhoneInterface { // 인터페이스 선언
    final int TIMEOUT = 10000; // 상수 필드 선언
    void sendCall();           // 추상 메소드
    void receiveCall();        // 추상 메소드
    default void printLogo() { // default 메소드
        System.out.println("** Phone **");
    }
}

interface MobilePhoneInterface extends PhoneInterface { // 인터페이스 상속
    void sendSMS();
    void receiveSMS();
    // MobilePhoneInterface 일정 및 수학함수 계산기가 포함된다고 하자
    // public int calculate(int x, int y); // PDA 클래스에서 구현하는 걸로 하자.
}

interface MP3Interface { // 인터페이스 선언
    public void play();
    public void stop();
}

class PDA { // 클래스 작성
    public int calculate(int x, int y) {
        return x + y;
    }
}
```

ISP의 위반 사례 해결

```
class FeaturePhone implements MobilePhoneInterface {
```

```
    @Override
```

```
    public void sendCall() {  
        System.out.println("띵동띵동띵동");  
    }
```

```
    @Override
```

```
    public void receiveCall() {  
        System.out.println("땡땡땡.");  
    }
```

```
    @Override
```

```
    public void sendSMS() {  
        System.out.println("봍");  
    }
```

```
    @Override
```

```
    public void receiveSMS() {  
        System.out.println("삐빅삐빅삐빅");  
    }
```

```
/* * 이제 구현하지 않아도 된다. *
```

```
    @Override
```

```
    public int calculate(int x, int y) {  
        return 0;  
    }
```

```
*/
```

```
}
```

ISP의 위반 사례 해결

```
class SmartPhone extends PDA implements MobilePhoneInterface, MP3Interface {
```

```
// MobilePhoneInterface의 추상 메소드 구현
```

```
@Override
```

```
public void sendCall() {  
    System.out.println("따르릉따르릉~~");  
}
```

```
@Override
```

```
public void receiveCall() {  
    System.out.println("전화 왔어요.");  
}
```

```
@Override
```

```
public void sendSMS() {  
    System.out.println("문자갑니다.");  
}
```

```
@Override
```

```
public void receiveSMS() {  
    System.out.println("문자왔어요.");  
}
```

```
/* * PDA에 그대로 상속하기 때문에 오버라이드 하지 않는다.
```

```
@Override
```

```
public int calculate(int x, int y) {  
    return x + y;  
}
```

```
*/
```

```
// MP3Interface의 추상 메소드 구현
```

```
@Override
```

```
public void play() {  
    System.out.println("음악 연주합니다.");  
}
```

```
@Override
```

```
public void stop() {  
    System.out.println("음악 중단합니다.");  
}
```

```
// 추가로 작성한 메소드
```

```
public void schedule() {  
    System.out.println("일정 관리합니다.");  
}
```

```
}
```

ISP의 위반 사례 해결

```
public class InterfaceEx {  
    public static void main(String [] args) {  
        SmartPhone phone = new SmartPhone();  
        phone.printLogo();  
        phone.sendCall();  
        phone.play();  
        System.out.println("3과 5를 더하면 " + phone.calculate(3,5));  
        phone.schedule();  
    }  
}
```



ISP 원칙 적용 주의 사항

객체지향설계와 패턴

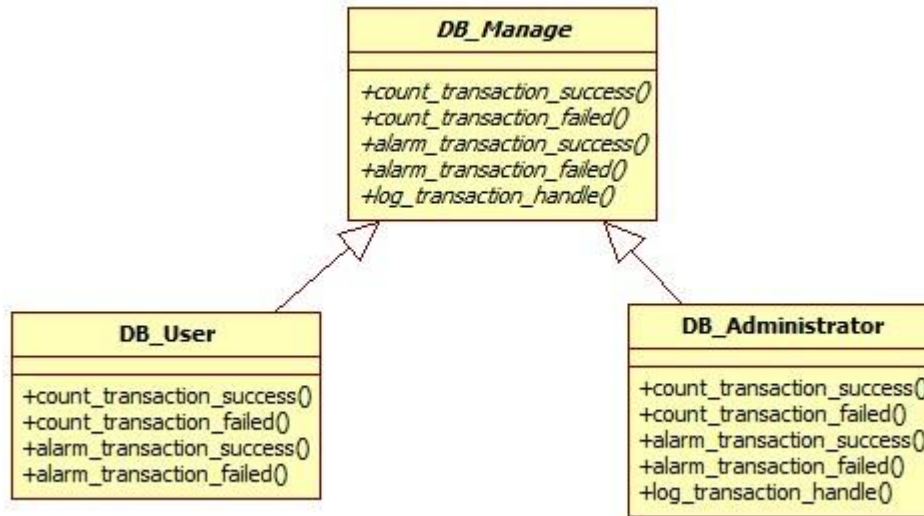
2026.03.24(Tue)



1. SRP와 ISP 원칙 사이의 관계

- 처음에는 별거 아닐지는 몰라도 SRP(단일책임원칙)과 ISP(인터페이스 분리 원칙) 사이에 약간 설계상의 문제가 될 수 있다.
 - 지금 해설은 어느 정도 해소가 된 내용들이라 그렇지 이후에 이렇게 설계가 없다거나 없어야 한다는 것은 아니다.
- SRP가 클래스의 단일 책임 원칙이라면, ISP는 인터페이스의 단일 책임 원칙이라고 볼 수 있다.
 - 이 말은 인터페이스의 기능을 책임에 맞게 추상 메소드를 설계하면 된다는 의미이다. 하지만 책임을 준수하더라도 실무에서는 ISP가 만족되지 않을 수 있는 사례가 존재한다.

1. SRP와 ISP 원칙 사이의 관계



위와 같이 DB_Manage 인터페이스엔 `count_transaction_success()`, `count_transaction_failed()`, `alarm_transaction_success()`, `alarm_transaction_failed()`, `log_transaction_handle()` 이라는 추상 메서드가 선언되어 있다. 이들은 트랜잭션 실행 성공여부 관리에 필요한 기능들이며 DB 관리 수준의 내용으로 단일 책임에 위배되지 않는다고 가정하자.

그런데 이를 구현하는 스키마 설계자와 같은 일반 사용자(DB_User)는 `log_transaction_handle()`에 대한 구현은 진행할 수 없기 때문에 ISP 위반으로 이어진다.

따라서 책임을 잘 구성해 놓은 것 같지만 실제 적용되는 클래스에는 부합되지 않을 수 있기 때문에 책임을 더 분리해야 옳을 수 있다.

정리하면 "SRP를 만족하면 ISP는 당연 성립되는가?" 라고 물어본다면 반드시 그렇다고는 볼 수 없다고 답하는 게 옳을 것이다.

2. 인터페이스 분리는 한번만 하자

- 이 얘기는 좀 억지같아 보이겠지만 인터페이스 분리를 2회 이상 수행할 필요가 있다면 클래스와 인터페이스들의 관계를 다시 살펴보고 각각의 클래스와 인터페이스의 내부를 다시 설계하는 것이 옳다.
 - SRP와 ISP의 성향의 차이로 클래스의 구조가 바뀔 수도 있지만 호출과 결과 수신 책임이 늘어나는 경우가 있다면 설계단계에서는 바로 리팩토링/재구조화를 하는 게 맞다.
 - 설계단계에서도 이런 정도인데 구현이 진행되거나 완료된 상황에서라면 유지보수를 시행하는 것이 좋을 수 있다.
 - 이미 구현되어 있는 프로젝트에 또 인터페이스들을 분리한다면 이미 해당 인터페이스를 구현한 모든 클래스들과 이를 사용하는 클라이언트(사용자)에서 문제가 일어날 수 있다.
 - 아니 단정적으로 말해 발생한다.

2. 인터페이스 분리는 한번만 하자

- 인터페이스는 한번 구성했으면 어지간해서 변하면 안되는 기초나 기본 정책 같은 것이다.
 - 우측통행이라거나 인화성 물질 주변에선 화기 주의와 같이 당연함을 설계자가 정하는 것이다.
- 따라서 처음 설계부터 기능의 변화를 생각해두고 인터페이스를 설계해야 하지만 현실적으로 어려운 부분으로 개발자(설계자)의 능력에 달렸다고 봐야 한다.
 - 인터페이스를 통해 구현을 강제할 부분과 상속을 통해 얻어오거나 변형할 내용을 정하는 것도 좀 어려운 부분일 수 있다.

결론

- ISP는 SRP와 차이가 있고 변경없는 견고한 설계는 상속에만 있는 것이 아니라 인터페이스를 이용하는 것에 큰 의미를 부여할 수 있다.

